

StyleSense – Generative AI Powered Fashion Recommendation System

Project Description:

StyleSense is a Generative AI-powered fashion recommendation and styling platform developed to help users make smarter and more confident fashion decisions. The application combines modern web technologies with advanced Artificial Intelligence models to provide personalized outfit recommendations, fashion insights, trend analysis, and product suggestions based on user preferences.

In today's digital era, users often struggle to select suitable outfits for different occasions, identify matching accessories, and stay updated with changing fashion trends. StyleSense addresses these challenges by acting as a virtual fashion assistant capable of understanding user requirements and generating intelligent recommendations. The platform enables users to upload fashion images, interact with an AI stylist chatbot, discover similar products, build complete outfits, and explore trending fashion styles.

The system integrates multiple AI technologies including Groq-powered LLaMA models, Google Gemini AI, and Hugging Face models to deliver fast, accurate, and context-aware recommendations. The application analyzes fashion-related inputs and generates suggestions based on clothing categories, color combinations, occasion requirements, budget constraints, and personal style preferences.

StyleSense is designed using a React-based frontend and a FastAPI backend, ensuring a responsive user experience and efficient communication with AI services. The project demonstrates how Generative AI can transform the fashion industry by providing intelligent styling assistance, enhancing user engagement, and simplifying fashion-related decision-making.

Scenarios

Scenario 1: AI Fashion Chat Assistant

A user interacts with the AI Stylist chatbot and requests outfit recommendations for various occasions such as college events, office meetings, parties, weddings, casual outings, or festive celebrations. The chatbot understands the user's requirements through natural language conversations and analyzes factors such as occasion, style preferences, budget, and fashion trends. Based on the user's input, the AI generates personalized styling recommendations and provides detailed fashion guidance. The chatbot acts as a virtual fashion consultant by helping users select appropriate outfits and accessories while ensuring style consistency and confidence.

AI generates:

- Outfit recommendations
- Color combinations and matching palettes
- Fashion and styling tips
- Accessory suggestions
- Footwear recommendations
- Occasion-based styling guidance
- Grooming and appearance suggestions
- Personalized fashion advice based on user preferences

Outcome:

The user receives a complete styling solution that helps them make informed fashion decisions without requiring professional fashion consultation.

Scenario 2: Fashion Image Analysis

A user uploads an image of a dress, shirt, footwear item, or any fashion-related product to the StyleSense platform. The system utilizes advanced AI-powered image analysis techniques to examine the uploaded image and extract important fashion attributes.

The platform analyzes various visual elements such as clothing type, color combinations, design patterns, fabric appearance, and overall style. Based on this analysis, the AI generates personalized styling insights and recommends matching fashion items that complement the uploaded outfit. The system also provides suggestions for accessories, footwear, and suitable occasions where the outfit can be worn.

The system performs:

- Clothing type identification
- Color and pattern analysis
- Style classification
- Fashion attribute extraction
- Matching outfit recommendations
- Accessory suggestions
- Footwear recommendations
- Occasion suitability analysis
- Personalized styling insights
-

Outcome:

The user receives a detailed analysis of the uploaded fashion item along with intelligent recommendations that help improve styling choices, discover matching products, and create complete fashion looks.

Scenario 3: Budget-Based Product Recommendation

A user enters a preferred budget range along with fashion requirements such as gender, clothing category, style preference, color choice, or occasion. The StyleSense platform analyzes the user's input and searches its fashion database to identify products that match the specified criteria.

The AI recommendation engine evaluates multiple factors including product category, style compatibility, current fashion trends, and price range before generating suitable recommendations. The system ensures that the suggested products align with the user's preferences while remaining within the selected budget. It also provides alternative options and complete outfit combinations to help users make better purchasing decisions.

The system provides:

- Budget-friendly fashion recommendations
- Product filtering based on user preferences
- Similar product suggestions
- Outfit combinations within budget
- Trend-based fashion recommendations
- Category-wise product recommendations
- Product descriptions and pricing details
- Shopping assistance for informed purchases

Outcome:

The user receives personalized fashion product recommendations that match their style preferences and budget constraints, enabling them to discover affordable and stylish clothing options while enhancing their overall shopping experience.

Architecture Overview

StyleSense follows a modern multi-layered client-server architecture designed to deliver personalized fashion recommendations, intelligent styling assistance, and fashion analysis through the integration of Generative AI technologies. The architecture consists of four major layers: Frontend Layer, Backend Layer, AI Service Layer, and Deployment Layer.

The Frontend Layer is developed using React.js, Vite, Tailwind CSS, and JavaScript. It provides an interactive and responsive user interface that allows users to access features such as AI Stylist Chat, Fashion Image Analysis, Outfit Builder, Similar Product Search, Fashion Trends Analysis, Color Palette Generator, Style Quiz, and User History.

The Backend Layer is implemented using FastAPI and Python. It handles user requests, manages API communication, processes uploaded images, generates recommendations, and coordinates interactions between the frontend and AI services.

The AI Service Layer integrates advanced Generative AI technologies including Groq-powered LLaMA models, Google Gemini AI, and Hugging Face models. These services are responsible for conversational assistance, outfit recommendation generation, fashion trend analysis, image understanding, and personalized styling suggestions.

The Deployment Layer ensures reliable application hosting and accessibility. The frontend application is deployed on Vercel, while the backend APIs are hosted on Render.

Architecture Components:

Frontend

- React.js
- Vite
- Tailwind CSS
- JavaScript

Backend

- FastAPI
- Python
- REST APIs

AI Services

- Groq (LLaMA 3)
- Google Gemini AI
- Hugging Face Models
- Generative AI APIs

Deployment

- Vercel (Frontend)
- Render (Backend)

This architecture enables StyleSense to deliver intelligent, scalable, and real-time fashion assistance while maintaining high performance, flexibility, and an enhanced user experience.

Core Technologies :

- **React.js**
React.js is a popular JavaScript library used for building dynamic and interactive user interfaces. It enables the development of reusable components, improves application performance through efficient rendering, and provides a seamless user experience across different devices and screen sizes.
- **Vite**
Vite is a modern frontend build tool that offers faster development and optimized production builds. It significantly reduces application loading time and improves the overall development workflow by providing instant server startup and efficient module replacement.
- **Tailwind CSS**
Tailwind CSS is a utility-first CSS framework used to create responsive and visually appealing user interfaces. It simplifies the styling process by providing pre-defined utility classes, enabling rapid UI development while maintaining consistency across the application.
- **FastAPI**
FastAPI is a high-performance Python web framework used for developing RESTful APIs. It supports asynchronous programming, automatic API documentation, and efficient request handling, making it suitable for scalable backend development.

- **Python**

Python serves as the primary programming language for implementing backend logic, API integrations, and AI functionalities. Its simplicity, extensive library support, and compatibility with machine learning frameworks make it ideal for developing intelligent applications.

- **Groq (LLaMA 3)**

Groq provides high-speed AI inference capabilities through the LLaMA 3 large language model. It is used to generate intelligent chatbot responses, fashion recommendations, and conversational styling guidance with low latency and high efficiency.

- **Google Gemini AI**

Google Gemini AI is utilized for advanced reasoning, content generation, and fashion-related recommendations. It helps analyze user queries, generate personalized outfit suggestions, and provide contextual fashion advice based on user preferences.

- **Hugging Face**

Hugging Face provides access to a wide range of pre-trained machine learning and natural language processing models. These models support various AI functionalities such as text processing, recommendation enhancement, and fashion-related analysis.

- **REST APIs**

REST APIs facilitate communication between the frontend and backend components of the application. They enable efficient data exchange, allowing different modules to interact seamlessly and provide real-time responses to user requests.

- **Render**

Render is used for deploying and hosting the backend services of the application. It provides a reliable cloud infrastructure that ensures continuous availability, scalability, and secure API access.

- **Vercel**

Vercel is used for deploying the frontend application and delivering content efficiently to users. It offers fast performance, automatic deployments, and global content delivery, ensuring a smooth user experience.

Software Requirements

- Python 3.10+
- Node.js
- npm
- **Code Editor:** VS Code or any preferred IDE
- Git
- **Groq API Key:** Obtain from <https://console.groq.com>

Knowledge Prerequisites

To understand and develop the StyleSense platform, users should have basic knowledge of programming, web development, API integration, and Artificial Intelligence concepts.

Python Basics

Knowledge of Python functions, classes, exception handling, and library usage is required for backend development and AI integration.

React.js

Understanding React components, state management, and event handling helps in developing the frontend user interface.

JavaScript

Familiarity with JavaScript concepts such as DOM manipulation, asynchronous programming, and Fetch API is useful for implementing interactive features.

API Integration

Basic knowledge of REST APIs, HTTP requests, and JSON data handling is required for communication between frontend, backend, and AI services.

HTML & CSS

Understanding HTML and CSS helps in designing responsive and user-friendly web interfaces.

Artificial Intelligence Basics

Basic knowledge of Generative AI, Natural Language Processing, and recommendation systems is beneficial for understanding the intelligent features of the application.

Git and GitHub

Familiarity with version control systems helps in managing source code and project collaboration efficiently.

Project Workflow :

Phase 1: Requirement Analysis:

- **Activity 1.1** : Study fashion recommendation systems and Generative AI applications.
- **Activity 1.2**: Define project objectives and architecture.

Phase 2: Backend Development:

- Activity 2.1: Create FastAPI server.
- Activity 2.2: Integrate Gemini API.
- Activity 2.3: Develop chatbot functionality.
- Activity 2.4: Implement image analysis module.
- Activity 2.5: Develop outfit recommendation engine.

Phase 3: Frontend Development

- **Activity 3.1:** Create React application.
- **Activity 3.2:** Develop responsive user interface.
- **Activity 3.3:** Integrate frontend with backend APIs.

Phase 4: Deployment

- **Activity 4.1:** Deploy backend using Render.
- **Activity 4.2:** Deploy frontend using Vercel.

Phase 5: Testing and Optimization

- **Activity 5.1:** Functional testing.
- **Activity 5.2:** Performance optimization.

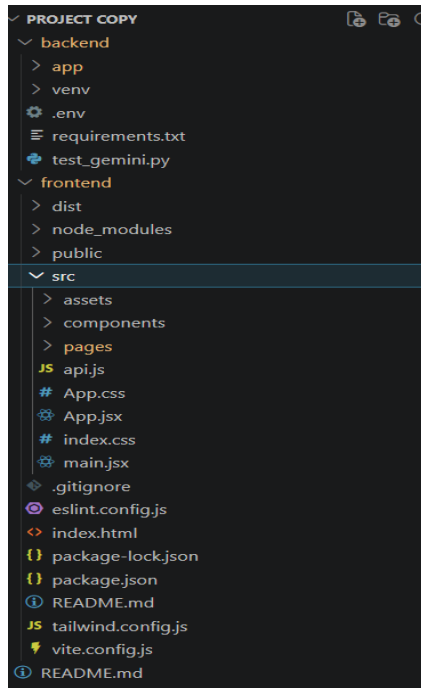
Technical Architecture :

The project structure is organized to separate frontend, backend, and AI functionalities into independent modules. This separation improves code readability, simplifies debugging, and enables collaborative development. Each module is responsible for a specific set of tasks, such as user interface management, API processing, recommendation generation, image analysis, and data handling. The organized structure ensures efficient development practices and supports the future expansion of the StyleSense platform with additional features and AI capabilities.

The technical architecture of StyleSense is designed using a modular and scalable approach to ensure efficient development, maintenance, and future enhancements. The application follows a client-server architecture where the frontend, backend, and AI services work together to provide personalized fashion recommendations and styling assistance.

The frontend is responsible for user interaction and displaying recommendations, while the backend handles business logic, API communication, and AI service integration. Multiple AI models are utilized to generate fashion insights, outfit suggestions, image analysis results, and trend recommendations. This architecture ensures high performance, flexibility, and seamless communication between all system components.

Project Structure Setup



MILESTONE 1: Environment Setup & Gemini API Configuration

Activity 1.1: Install Dependencies

- **Frontend:** npm install
- **Backend:** pip install -r requirements.txt

Activity 1.2: Configure Gemini API

Create .env file: YOUR_API_KEY



MILESTONE 2: Core Backend Development

Build FastAPI backend services for:

- AI chatbot
- Outfit recommendation
- Fashion trend analysis
- Image analysis
- Product recommendation

Import Libraries

```
import os
import re
import json
import urllib.parse
from typing import List, Dict, Any, Optional
from groq import Groq
import google.generativeai as genai
from app.config import settings
from app.database import get_all_products, PRODUCTS_DB

# Initialize Groq Client
groq_client = None
if settings.GROQ_API_KEY and "your_" not in settings.GROQ_API_KEY:
    try:
        groq_client = Groq(api_key=settings.GROQ_API_KEY)
    except Exception as e:
        print(f"Error initializing Groq client: {e}")

# Initialize Gemini Client
gemini_initialized = False
if settings.GEMINI_API_KEY and "your_" not in settings.GEMINI_API_KEY:
    try:
        genai.configure(api_key=settings.GEMINI_API_KEY)
        gemini_initialized = True
    except Exception as e:
        print(f"Error configuring Gemini client: {e}")

# --- STRICT CHATBOT CLASSIFICATION & IMPLEMENTATION ---

STRICT_NON_FASHION_REPLY = "I am StyleSense AI and can only assist with fashion and

def is_fashion_query(query: str) -> bool:
    """
```

```

d > app > services.py > ...
def is_fashion_query(query: str) -> bool:
    # Allow simple greetings and questions about the bot itself to pass
    query_clean = query.strip().lower().rstrip("?!")
    greetings = {
        "hi", "hello", "hey", "hola", "greetings", "good morning", "good afternoon", "good evening",
        "who are you", "what is your name", "what are you", "how are you", "how's it going", "what can you do",
        "help", "help me", "info", "about", "stylesense", "style sense"
    }
    if query_clean in greetings or any(query_clean.startswith(g) for g in ["hi ", "hello ", "hey ", "who are you", "what ca
    return True

    if not groq_client:
        # Fallback keyword classification if Groq is not available
        fashion_keywords = [
            "fashion", "wear", "dress", "shirt", "pants", "kurta", "kurti", "outfit", "style", "styling",
            "look", "color", "match", "shoe", "sneaker", "sandal", "accessory", "bag", "watch", "sunglass",
            "trend", "wardrobe", "jacket", "hoodie", "jeans", "chinos", "cargo", "wear", "suit", "beauty", "cosmetics"
        ]
        query_lower = query.lower()
        return any(kw in query_lower for kw in fashion_keywords)

    try:
        prompt = f"""
        Determine if the user query is about fashion, clothing, footwear, accessories, beauty, makeup, styling, matching co

        Examples of FASHION:
        - "suggest a blue shirt pairing"
        - "what is trending this summer?"
        - "are sneakers ok with formal pants?"
        - "hi, help me find a kurti"
        - "how to style beige chinos"

        Examples of NON_FASHION:
        - "what is the capital of India?"
        - "how do I sort a list in python?"
        - "tell me a joke"

```

```

d > app > services.py > ...
def is_fashion_query(query: str) -> bool:
    """
    Reply with exactly 'FASHION' or 'NON_FASHION'. Output absolutely nothing else. No punctuation, no explanation.
    """
    chat_completion = groq_client.chat.completions.create(
        messages=[{"role": "user", "content": prompt}],
        model="llama-3.1-8b-instant",
        temperature=0.0,
        max_tokens=5
    )
    result = chat_completion.choices[0].message.content.strip().upper()
    return "FASHION" in result
except Exception as e:
    print(f"Classification failed, falling back to True: {e}")
    return True

def chat_with_groq(message: str, history: List[Dict[str, str]]) -> str:
    """
    Handles chatbot conversations with strict enforcement of the fashion-only query rule.
    """
    # 1. Enforce Strict Fashion Rule
    if not is_fashion_query(message):
        return STRICT_NON_FASHION_REPLY

    if not groq_client:
        return "Chat service is temporarily offline (Missing GROQ_API_KEY), but I can still assist with local fashion catalog."

    # 2. Structure Fashion Dialogue
    system_prompt = """
    You are StyleSense AI, an elite personal stylist specializing in Indian fashion with deep knowledge of budget-friendly styling.
    Your personality: Warm, specific, practical – like a knowledgeable friend who knows fashion inside out.

    CRITICAL Response Rules:
    1. ALWAYS suggest complete outfits with SPECIFIC items, colors, and approximate INR prices.
       Example: "White cotton A-line kurti (₹400-600, available on Myntra/Amazon) + straight-fit black palazzo (₹300-500) + gold
    2. ALWAYS mention estimated price ranges in Indian Rupees (₹) for every item you suggest.

```

MILESTONE 3: Frontend Development – UI/UX Design

Activity 3.1: Main Interface

Key Features:

- AI Stylist Chatbot
- Outfit Recommendation
- Fashion Image Analysis
- Product Search
- Budget Filter
- Trend Explorer
- Color Palette Explorer
- Responsive Design

Components :

- Home Page
- Chat Assistant
- Outfit Recommendation Page
- Image Analysis Module
- Product Recommendation Page
- Fashion Trends Page

```

// src/pages/analysis/analysis.jsx
import React, { useCallback, useState } from 'react';
import { useDropzone } from 'react-dropzone';
import { motion, AnimatePresence } from 'framer-motion';
import {
  FiUploadCloud, FiCheck, FiStar, FiShoppingBag,
  FiX, FiRefreshCw, FiExternalLink, FiGrid, FiTrendingUp, FiSettings, FiShare2
} from 'react-icons/fi';
import toast, { Toaster } from 'react-hot-toast';
import api from '../../api';
import { ScoreRing, Badge } from '../../components/UI';

/* ----- Step Labels for the progress indicator ----- */
const STEP_LABELS = ['Category', 'Upload Photo', 'Set Budget', 'Analyze'];

function StepIndicator({ current }) {
  return (
    <div className="flex items-center justify-center gap-8 mb-10">
      {STEP_LABELS.map((label, i) => {
        const done = i < current;
        const active = i === current;
        return (
          <div key={i} className="flex items-center">
            <div className="flex flex-col items-center">
              <div className={`w-9 h-9 rounded-full flex items-center justify-center text-sm font-bold transition-all duration-300 ${
                done
                  ? 'bg-gradient-to-br from-gold-500 to-gold-600 text-slate-950 shadow-lg shadow-gold-500/30'
                  : active
                  ? 'bg-gradient-to-br from-gold-500 to-gold-600 text-slate-950 shadow-lg shadow-gold-500/30 ring-4 ring-gold-500/20'
                  : 'bg-white/5 text-slate-500'
              }`} />
            {done ? <FiCheck size={15} /> : i + 1}
          </div>
          <span className={`text-xs mt-1.5 font-medium ${
            active ? 'text-gold-400' : 'text-slate-500'
          }`} />
            {label}
          </span>
        </div>
      )}
      {i < STEP_LABELS.length - 1 && (
        <div className={`w-16 h-0.5 mx-1 mb-5 transition-colors duration-300 ${
          i < current
            ? 'bg-gradient-to-r from-gold-500 to-gold-600'
            : 'bg-white/5'
          }`} />
        )}
      </div>
    </div>
  );
}

```

```

2 src > pages > W ProductsPage.jsx 7 ...
import React, { useState } from 'react';
import ProductCard from '../components/ProductCard';
import Loader from '../components/Loader';
import api from '../api';
import { FiSliders, FiDollarSign, FiSearch, FiAlertCircle, FiGrid } from 'react-icons/fi';

export default function ProductsPage() {
  const [gender, setGender] = useState("Women's Fashion");
  const [category, setCategory] = useState('Topwear');
  const [detectedItem, setDetectedItem] = useState('');
  const [colorsInput, setColorsInput] = useState('Black');
  const [style, setStyle] = useState('Casual');
  const [budget, setBudget] = useState(1500);
  const [products, setProducts] = useState([]);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState(null);
  const [searched, setSearch] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    setProducts([]);
    setSearch(true);

    try {
      // Split colors by comma and clean whitespace
      const colors = colorsInput.split(',').map(c => c.trim()).filter(Boolean);
      const response = await api.getSimilarProducts(
        gender,
        category,
        detectedItem || category, // fallback to category if empty
        colors,
        style,
        budget
      );
      setProducts(response.matches || []);
    } catch (err) {
      console.error(err);
      setError(typeof err === 'string' ? err : 'Failed to search products. Please try again.');
```

```

2 src > pages > W OutfitHistoryPage.jsx 7 ...
import React, { useState, useEffect } from 'react';
import { motion } from 'framer-motion';
import { FiTrash2, FiClock, FiShoppingBag } from 'react-icons/fi';

export default function OutfitHistoryPage() {
  const [history, setHistory] = useState([]);

  useEffect(() => {
    const saved = localStorage.getItem('stylesense_history');
    if (saved) setHistory(JSON.parse(saved));
  }, []);

  const clearAll = () => {
    localStorage.removeItem('stylesense_history');
    setHistory([]);
  };

  const deleteOne = (idx) => {
    const updated = history.filter((_, i) => i !== idx);
    localStorage.setItem('stylesense_history', JSON.stringify(updated));
    setHistory(updated);
  };

  return (
    <div className="w-full py-8 px-4 max-w-4xl mx-auto space-y-8">
      <div className="text-center">
        <span className="inline-block px-4 py-1.5 rounded-full bg-gold-500/10 text-gold-400 text-[10px] font-extrabold tracking-widest">
          <h1 className="text-3xl font-black tracking-tight mb-2">Outfit History</h1>
          <p className="text-sm text-slate-400">All your previously analyzed outfits</p>
        </div>

        {history.length > 0 && (
          <div className="flex justify-end">
            <button onClick={clearAll} className="flex items-center gap-2 px-4 py-2 rounded-xl border border-red-500/20 text-red-400 tex">
              <FiTrash2 size={12} /> Clear All
            </button>
          </div>
        )}

        {history.length === 0 ? (
          <div className="text-center py-20 glassmorphism border border-white/5 rounded-2xl">
            <FiClock className="text-4xl text-slate-600 mx-auto mb-4" />
            <p className="font-bold text-slate-500">No outfit history yet</p>
            <p className="text-sm text-slate-600 mt-1">Analyze an outfit in Image Analysis to see it here</p>
          </div>
        ) : (
          <div className="grid grid-cols-1 md:grid-cols-2 gap-5">
            {history.map((item, i) => (
              <motion.div key={i} initial={{ opacity: 0, y: 16 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: i * 0.05 }}>
                <div className="glassmorphism border border-white/5 rounded-2xl p-5 relative">

```

MILESTONE 4: Deployment

Activity 4.1:

Backend Deployment: Deploy backend on Render.

Deployment Settings:

Runtime: Python

Build Command:

pip install -r requirements.txt

Start Command

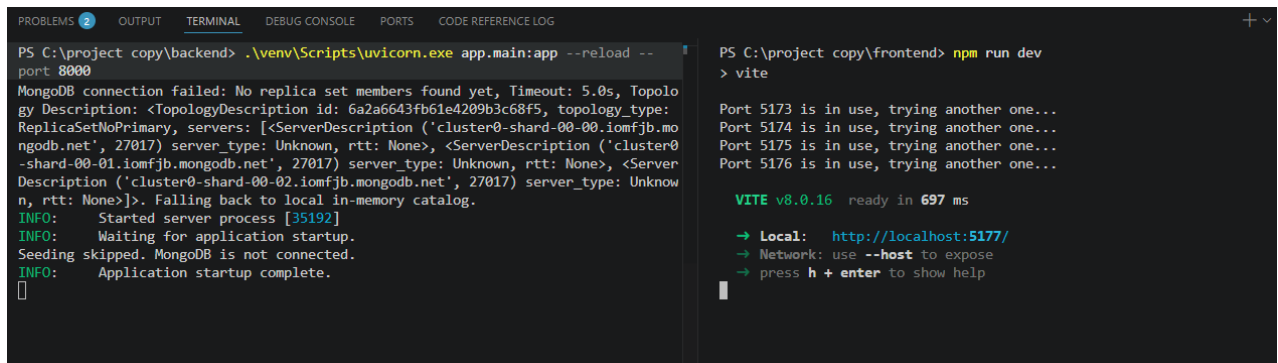
uvicorn app.main:app --host 0.0.0.0 --port \$PORT

Activity 4.2:

Frontend Deployment: Deploy frontend on Vercel.

Build Command

npm run build



```
PS C:\project copy\backend> .\venv\Scripts\uvicorn.exe app.main:app --reload --port 8000
MongoDB connection failed: No replica set members found yet, Timeout: 5.0s, Topology Description: <TopologyDescription id: 6a2a6643fb61e4209b3c68f5, topology_type: ReplicaSetNoPrimary, servers: [<ServerDescription ('cluster0-shard-00-00.iomfjb.mongodb.net', 27017) server_type: Unknown, rtt: None>, <ServerDescription ('cluster0-shard-00-01.iomfjb.mongodb.net', 27017) server_type: Unknown, rtt: None>, <ServerDescription ('cluster0-shard-00-02.iomfjb.mongodb.net', 27017) server_type: Unknown, rtt: None>]. Falling back to local in-memory catalog.
INFO: Started server process [35192]
INFO: Waiting for application startup.
Seeding skipped. MongoDB is not connected.
INFO: Application startup complete.
█

PS C:\project copy\frontend> npm run dev
> vite
Port 5173 is in use, trying another one...
Port 5174 is in use, trying another one...
Port 5175 is in use, trying another one...
Port 5176 is in use, trying another one...
VITE v8.0.16 ready in 697 ms
→ Local: http://localhost:5177/
→ Network: use --host to expose
→ press h + enter to show help
█
```

MILESTONE 5: Testing & Optimization

Activity 5.1: Functional Testing

login page:

The Login Page serves as the secure entry point to the StyleSense platform. It is designed with a modern and visually appealing user interface that allows users to access personalized fashion services and AI-powered recommendations. The page provides both Sign In and Sign Up options, enabling new users to create an account and existing users to securely access their profiles.

The interface includes email and password authentication, password visibility controls, a "Remember Me" option for user convenience, and a "Forgot Password" feature for account recovery. Additionally, users can access the platform through Google Authentication or continue using Guest Access without creating an account.

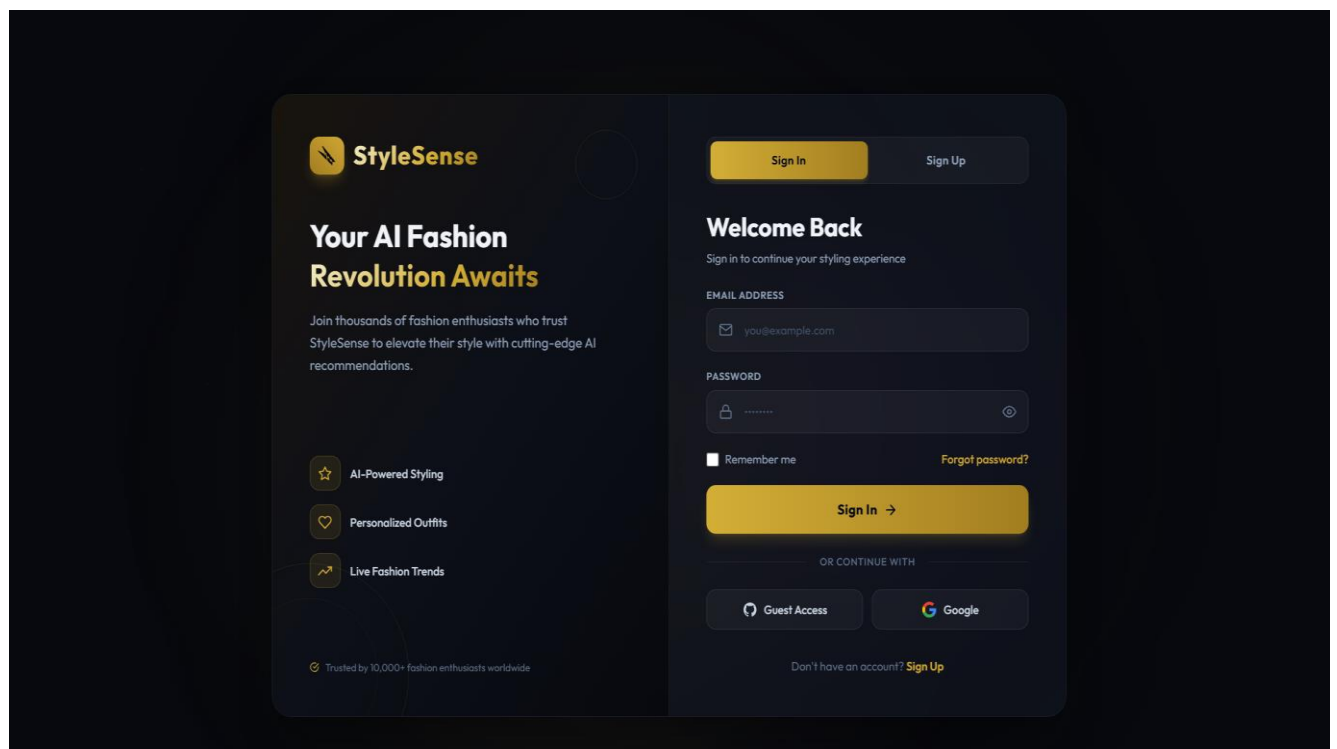
The left section of the page highlights the core benefits of the platform, including AI-Powered Styling, Personalized Outfits, and Live Fashion Trends. This helps users understand the key functionalities offered by StyleSense before entering the application.

Key Features:

- Secure user authentication system
- Sign In and Sign Up functionality
- Email and password-based login
- Password visibility toggle
- Remember Me option
- Forgot Password support
- Google Authentication integration
- Guest Access functionality
- Responsive and user-friendly interface
- Modern UI with branding and feature highlights

Outcome:

The Login Page ensures secure and convenient access to the StyleSense platform while providing users with multiple authentication options and a seamless onboarding experience.



Home page:

The Home Page serves as the main dashboard of the StyleSense platform and provides users with quick access to all major fashion assistance features. It is designed with a modern and visually appealing interface that introduces users to the capabilities of the platform while enabling seamless navigation across different modules.

The page highlights the core objective of StyleSense, which is to provide AI-powered fashion recommendations, outfit analysis, and personalized styling assistance. Users can directly access features such as AI Stylist Chat, Image Analysis, Outfit Builder, Similar Product Search, Fashion Trends, Color Palette Generator, Style Quiz, and History Management through the navigation menu.

The homepage includes prominent action buttons such as Analyze Your Outfit and Consult AI Stylist, allowing users to quickly begin their fashion journey. Feature cards displayed on the page provide a brief overview of the platform's primary functionalities, helping users understand the available services at a glance.

Key Features:

- Modern and responsive user interface
- Easy navigation to all platform modules
- AI Stylist Chat access
- Fashion Image Analysis
- AI Outfit Builder
- Similar Product Search
- Fashion Trends and Insights
- Color Palette Generator
- Style Quiz and Personalization
- User History Management
- Quick action buttons for faster access
- Mobile-friendly design

Outcome:

The Home Page acts as the central hub of the StyleSense platform, providing users with an intuitive starting point to explore AI-powered fashion recommendations, styling assistance, outfit analysis, and personalized fashion experiences.



Image Analysis:

The Image Analysis module is one of the core features of the StyleSense platform. It enables users to upload fashion-related images such as dresses, shirts, jackets, footwear, or complete outfits and receive AI-powered fashion insights. The module utilizes advanced image understanding capabilities combined with Generative AI technologies to analyze visual characteristics and generate personalized recommendations.

The image analysis process begins with gender selection, followed by image upload. Once the image is submitted, the system extracts important fashion attributes such as clothing category, dominant colors, style type, patterns, and overall outfit aesthetics. The AI then evaluates the uploaded fashion item and generates detailed styling insights based on current fashion trends and styling principles.

After the analysis is completed, users can specify a budget range and receive personalized product recommendations that complement the uploaded outfit. The system also suggests matching accessories, footwear options, color combinations, and styling improvements to create a complete and fashionable look.

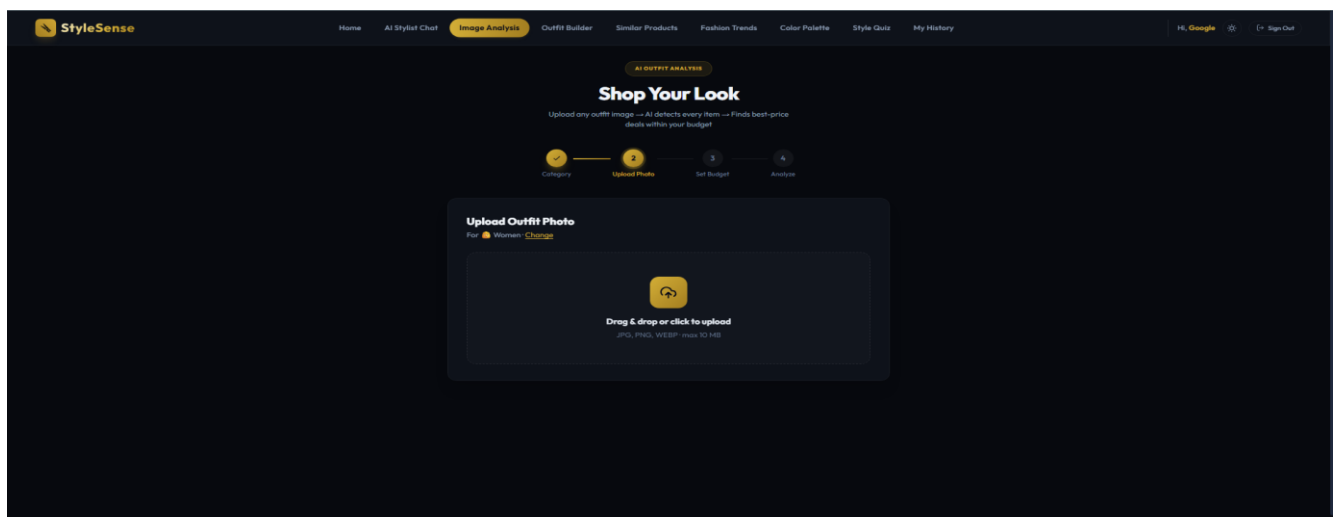
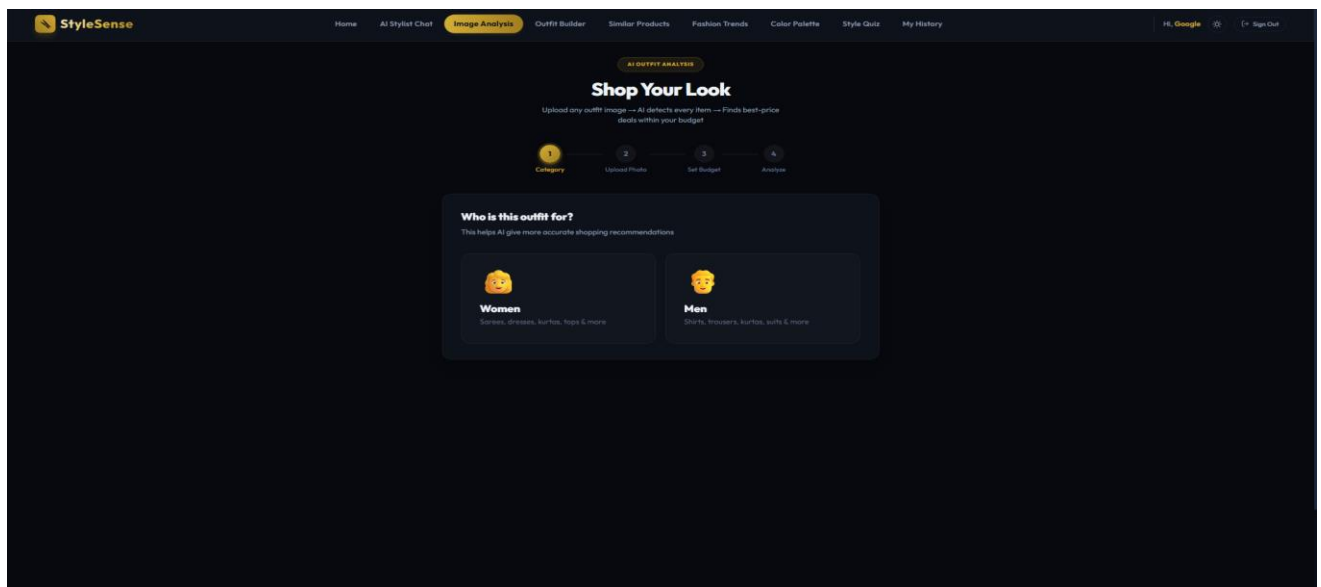
The Image Analysis module helps users discover new styling ideas, improve outfit coordination, and make informed fashion purchasing decisions through intelligent AI-driven recommendations.

Key Features:

- Fashion image upload and processing
- Gender-based outfit analysis
- Clothing category identification
- Color and style recognition
- Fashion attribute extraction
- Personalized styling recommendations
- Budget-based product suggestions
- Matching accessories and footwear recommendations
- AI-powered fashion insights
- User-friendly step-by-step workflow

Outcome:

The Image Analysis module transforms a simple clothing image into actionable fashion recommendations, enabling users to enhance their personal style, discover matching products, and create complete outfit combinations with the assistance of Artificial Intelligence.




[Home](#)
[AI Stylist Chat](#)
[Image Analysis](#)
[Outfit Builder](#)
[Similar Products](#)
[Fashion Trends](#)
[Color Palette](#)
[Style Quiz](#)
[My History](#)

Hi, Google

[Sign Out](#)

AI OUTFIT ANALYSIS

Shop Your Look

Upload any outfit image → AI detects every item → Finds best-price deals within your budget

Category

Upload Photo

Set Budget

Analyze

Set Your Budget



Outfit uploaded ✓

For Women

Under ₹500

Under ₹1,000

Under ₹2,000

Under ₹5,000

Enter custom amount

What is this garment? (e.g. Co-ord Set, Maxi Dress, Kurti)

Analyze & Shop Within ₹...


[Home](#)
[AI Stylist Chat](#)
[Image Analysis](#)
[Outfit Builder](#)
[Similar Products](#)
[Fashion Trends](#)
[Color Palette](#)
[Style Quiz](#)
[My History](#)

Hi, Google

[Sign Out](#)

Women's Elegant Fashion Garment

[New](#)
[Share](#)



8

Fashion Score

FORMAL

DRESSES

WOMEN

FASHION DETAILS

[Dress](#)
[V-Neck](#)
[Sleeveless](#)
[Maxi](#)
[Solid](#)
[Regular Fit](#)
[Georgette](#)
[Evening Wear](#)

DETECTED COLORS

[Black](#)

✓ STRENGTHS

- Elegant silhouette suitable for formal occasions
- Versatile color that pairs with many accessories

⚠ IMPROVEMENTS

- Add a statement belt to define the waist
- Layer with a blazer for office wear

Styling Coordinations

BOTTOMS

Not applicable for full-length dress

FOOTWEAR

Strappy Heels
Black Heel Sandals

ACCESSORIES

Clutch Bag
Gold Hoop Earrings

Analyze Another Outfit

AI Outfit Builder:

The AI Outfit Builder is an intelligent fashion recommendation module that helps users create complete outfit combinations based on their preferences, occasion, and budget. This feature utilizes Generative AI technologies to analyze user inputs and generate personalized outfit suggestions that align with current fashion trends and styling principles.

Users can specify details such as gender, occasion, preferred style, and budget range. Based on these inputs, the AI generates multiple outfit combinations consisting of clothing items, footwear, and accessories that complement each other. The recommendations are carefully curated to ensure color harmony, style consistency, and suitability for the selected occasion.

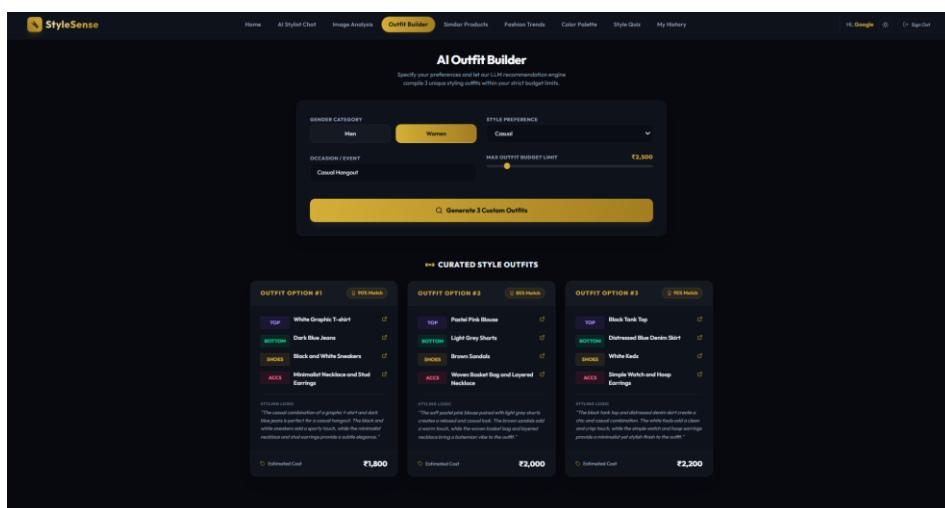
The module acts as a virtual personal stylist by helping users discover fashionable outfit ideas without spending time searching through multiple products. It simplifies the outfit selection process and provides users with ready-to-wear styling solutions tailored to their individual preferences.

Key Features:

- AI-powered outfit generation
- Occasion-based recommendations
- Budget-friendly outfit suggestions
- Multiple outfit combinations
- Personalized fashion recommendations
- Matching clothing and accessory suggestions
- Trend-aware styling guidance
- User-friendly outfit customization

Outcome:

The AI Outfit Builder enables users to quickly create stylish and coordinated outfits for different occasions, helping them save time while enhancing their overall fashion experience through intelligent and personalized recommendations.



Similar Product Search:

The Similar Product Search module helps users discover fashion products that closely match their preferences and style requirements. This feature utilizes AI-powered recommendation techniques to analyze user-selected criteria such as product category, color, style, occasion, and budget range to identify suitable fashion items.

Users can specify their desired fashion attributes and search preferences through an intuitive interface. The system then compares these inputs with available fashion products and generates a list of similar items that align with the user's requirements. The recommendations are designed to help users find alternative products, explore different styling options, and make informed purchasing decisions.

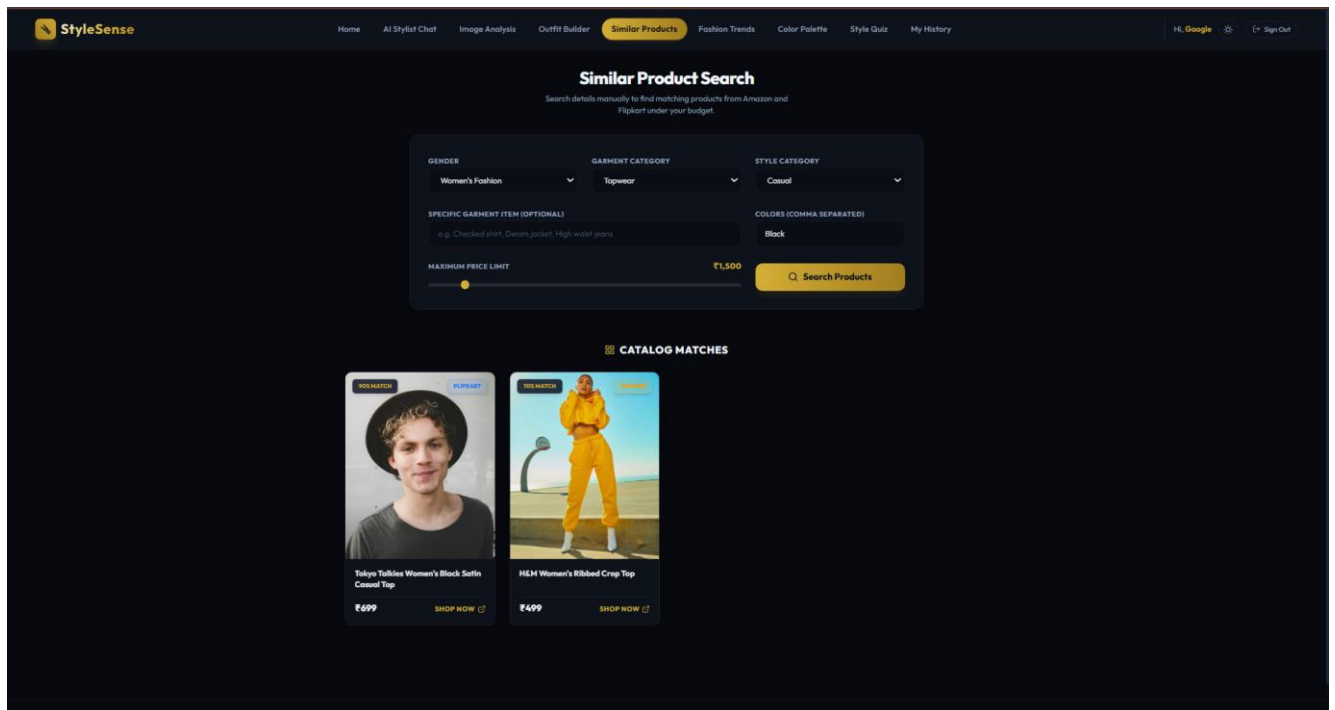
The module also considers current fashion trends and style compatibility while generating recommendations, ensuring that users receive relevant and fashionable product suggestions. This feature enhances the shopping experience by reducing search effort and providing personalized product discovery.

Key Features:

- AI-powered product recommendation system
- Similar fashion item discovery
- Category-based product search
- Color and style matching
- Budget-based product filtering
- Trend-aware recommendations
- Personalized shopping assistance
- User-friendly search interface
- Alternative product suggestions
- Fashion-focused recommendation engine

Outcome:

The Similar Product Search module enables users to efficiently discover fashion products that match their preferences, budget, and style requirements, making the shopping experience faster, smarter, and more personalized.



Fashion Trends & Analysis:

The Fashion Trends & Analysis module helps users stay updated with the latest developments in the fashion industry. This feature utilizes Artificial Intelligence and trend analysis techniques to provide insights into popular clothing styles, trending color palettes, seasonal fashion preferences, and emerging fashion patterns.

The module analyzes fashion data and presents valuable information that assists users in making informed styling decisions. It highlights current fashion trends, recommended outfit combinations, popular colors, and occasion-specific styling suggestions. Users can explore trend reports and fashion insights that align with modern fashion standards and consumer preferences.

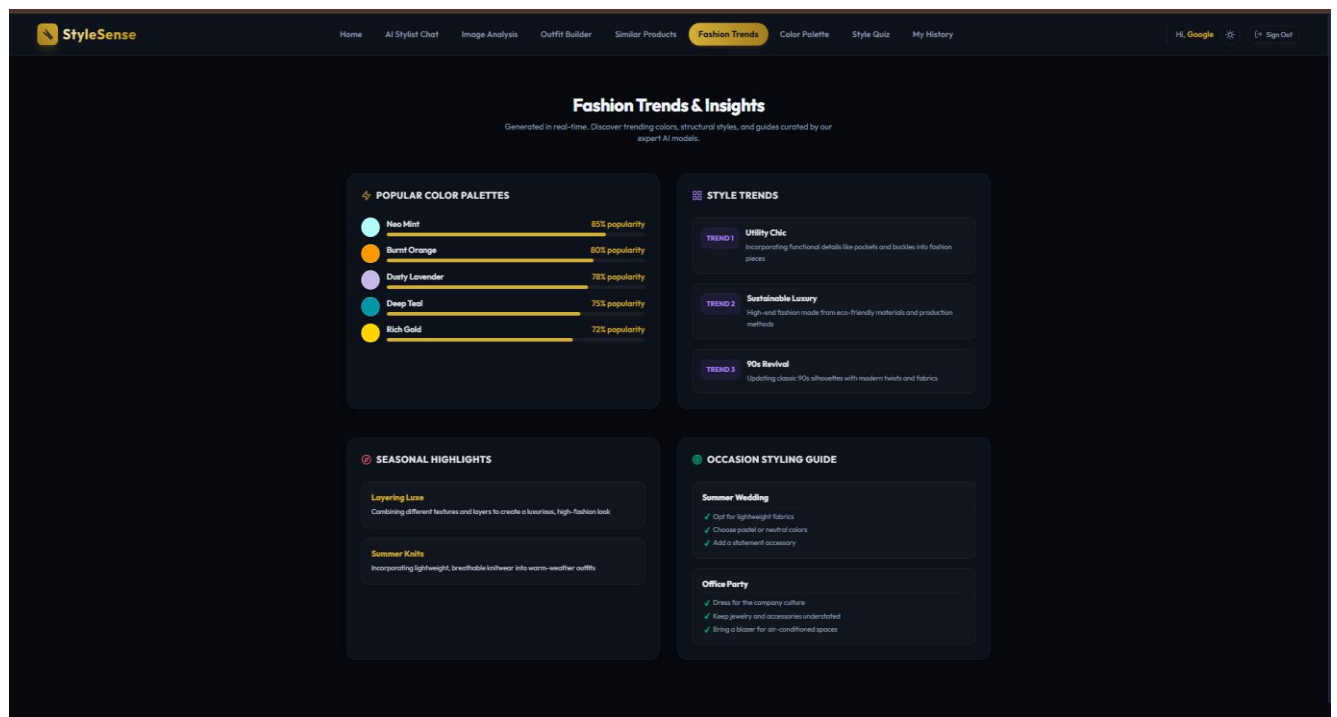
By providing real-time fashion intelligence, this feature enables users to adapt their wardrobe according to changing trends while maintaining their personal style. The module acts as a digital fashion guide that helps users understand what is currently popular and how to incorporate trending elements into their outfits.

Key Features:

- AI-powered fashion trend analysis
- Popular color palette recommendations
- Seasonal fashion insights
- Trending outfit suggestions
- Occasion-based styling guidance
- Emerging fashion trend identification
- Style preference analysis
- Fashion industry insights
- Personalized trend recommendations
- Interactive trend exploration

Outcome:

The Fashion Trends & Analysis module enables users to stay informed about the latest fashion movements, discover trending styles, and make confident styling decisions based on current industry trends and AI-generated fashion insights.



Colour Palette Generator:

The Colour Palette Generator is an AI-powered feature that helps users discover visually appealing color combinations for clothing and fashion accessories. This module analyzes user preferences and fashion trends to generate personalized color palettes that enhance overall outfit appearance and styling consistency.

Color coordination plays a crucial role in fashion and personal styling. Choosing the right combination of colors can significantly improve the visual appeal of an outfit and enhance an individual's overall appearance. The Colour Palette Generator simplifies this process by automatically suggesting color combinations that work well together. Users can experiment with different palettes and explore new fashion possibilities without requiring extensive knowledge of color theory. The feature also helps users understand how colors influence fashion aesthetics and supports the creation of balanced, attractive, and trend-conscious outfits.

The system provides complementary, analogous, monochromatic, and contrasting color combinations that can be used to create balanced and fashionable outfits. The generated palettes help users understand how different colors work together and assist in selecting clothing items that create a harmonious look.

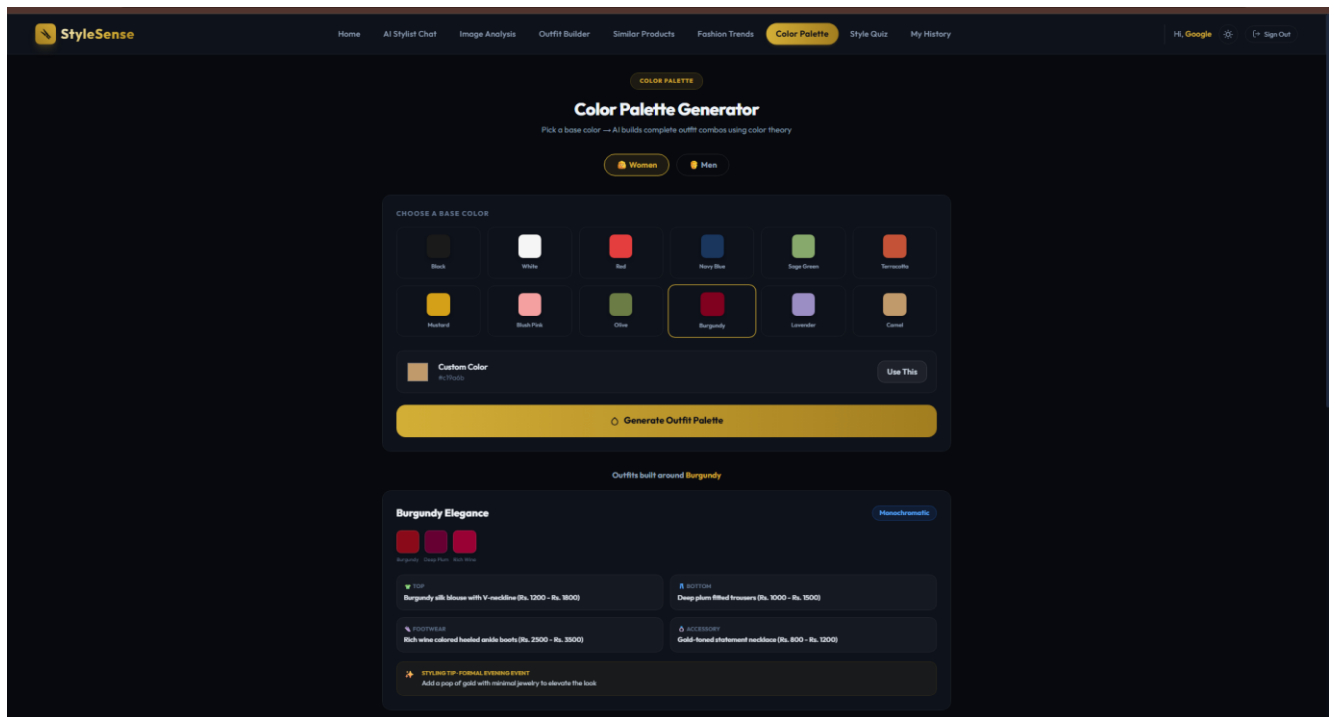
The Colour Palette Generator acts as a virtual color consultant by simplifying color selection and helping users build outfits with confidence. It is particularly useful for users who want to experiment with new styles, improve color coordination, and stay aligned with current fashion trends.

Key Features:

- AI-generated color palette recommendations
- Fashion-focused color combinations
- Complementary and contrasting color suggestions
- Seasonal color trend analysis
- Personalized color selection guidance
- Outfit color coordination assistance
- Trend-aware palette generation
- User-friendly and interactive interface
- Modern fashion color recommendations
- Styling enhancement through color harmony

Outcome:

The Colour Palette Generator helps users create stylish and well-coordinated outfits by providing intelligent color recommendations, improving fashion choices, and enhancing overall visual appeal through effective color matching.



Style Quiz:

The Style Quiz is an interactive feature designed to understand the user's fashion preferences, personality, and styling interests. Through a series of carefully designed questions, the system gathers information about the user's preferred clothing styles, favorite colors, fashion goals, lifestyle, and shopping habits.

The quiz helps the platform build a personalized fashion profile for each user. Based on the responses provided, the AI analyzes individual preferences and generates customized fashion recommendations that align with the user's unique style. This allows the platform to deliver more accurate outfit suggestions, color palettes, and product recommendations.

Every individual has a unique fashion sense, and understanding personal style is essential for providing meaningful recommendations. The Style Quiz acts as a personalization tool that enables the system to learn about the user's tastes and preferences. By identifying style patterns and fashion interests, the platform can recommend outfits that reflect the user's personality while also introducing new trends and styling ideas. This creates a more engaging and personalized fashion experience.

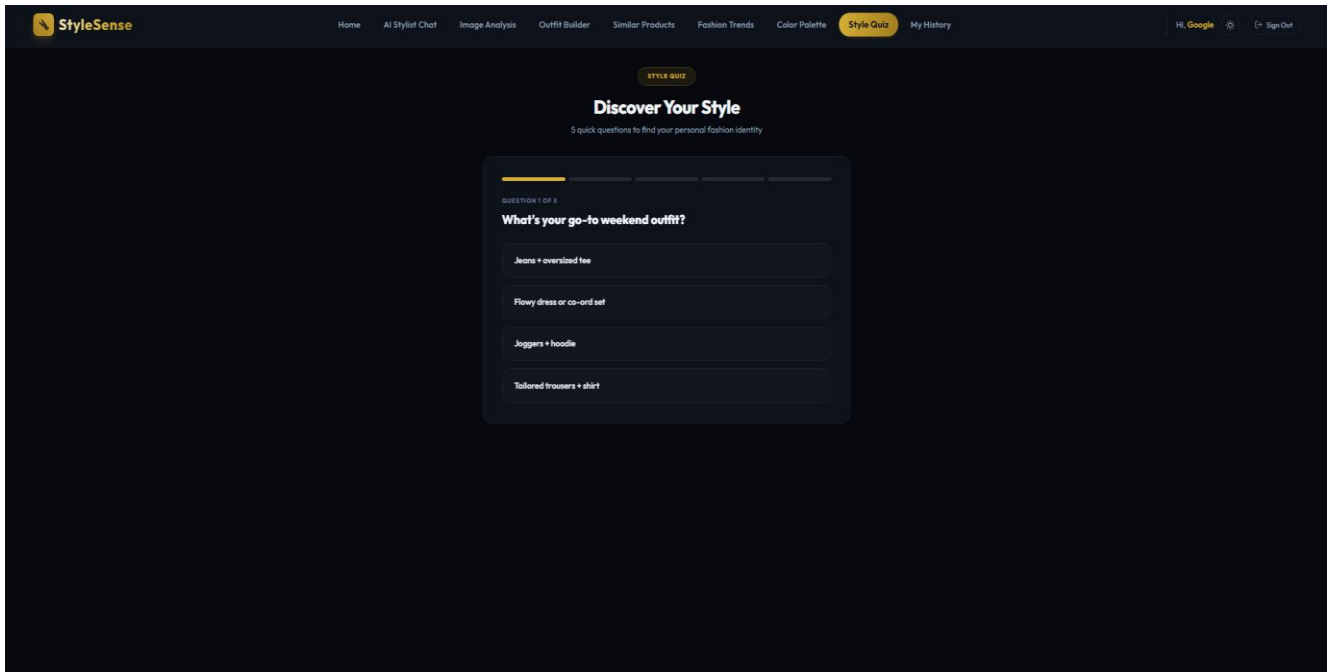
The Style Quiz not only improves recommendation accuracy but also helps users discover their fashion identity and explore styles they may not have considered before. It serves as an important component in delivering a tailored and user-centric fashion assistance experience.

Key Features:

- Interactive fashion personality assessment
- Personalized style profiling
- AI-driven recommendation enhancement
- Fashion preference analysis
- Color and outfit preference identification
- Lifestyle-based style suggestions
- Customized fashion insights
- Improved recommendation accuracy
- User engagement and personalization
- Easy-to-use questionnaire interface

Outcome:

The Style Quiz helps users better understand their fashion preferences while enabling the StyleSense platform to generate highly personalized outfit recommendations, styling advice, and fashion suggestions tailored to individual tastes and lifestyles.



The screenshot shows the 'StyleSense' website interface. The top navigation bar includes links for Home, AI Stylist Chat, Image Analysis, Outfit Builder, Similar Products, Fashion Trends, Color Palette, Style Quiz (highlighted), and My History. On the right, there are links for 'Hi, Google', a search icon, and 'Sign Out'. The main content area is titled 'Discover Your Style' with the subtitle '5 quick questions to find your personal fashion identity'. Below this, a quiz card displays 'QUESTION 1 OF 5' and the question 'What's your go-to weekend outfit?'. Four options are listed in a vertical stack: 'Jeans + oversized tee', 'Flowy dress or co-ord set', 'Joggers + hoodie', and 'Tailored trousers + shirt'.

History:

The History module allows users to view and manage their previous activities within the StyleSense platform. This feature stores records of past outfit recommendations, image analysis results, style quiz outcomes, product searches, and AI stylist interactions. By maintaining a history of user activities, the platform enables users to revisit and review their earlier fashion insights whenever needed.

The module serves as a personal fashion archive that helps users track their styling journey over time. Users can easily access previously generated recommendations, compare different outfit suggestions, and monitor changes in their fashion preferences. This functionality enhances user convenience by eliminating the need to repeat searches or analyses that have already been performed.

Fashion preferences often evolve over time, and users may wish to refer back to previous recommendations before making styling or purchasing decisions. The History module provides a centralized location where all past interactions are stored and organized. This allows users to maintain continuity in their fashion experience and make better-informed decisions based on their previous styling choices. The feature also improves personalization by helping the platform understand recurring preferences and user interests.

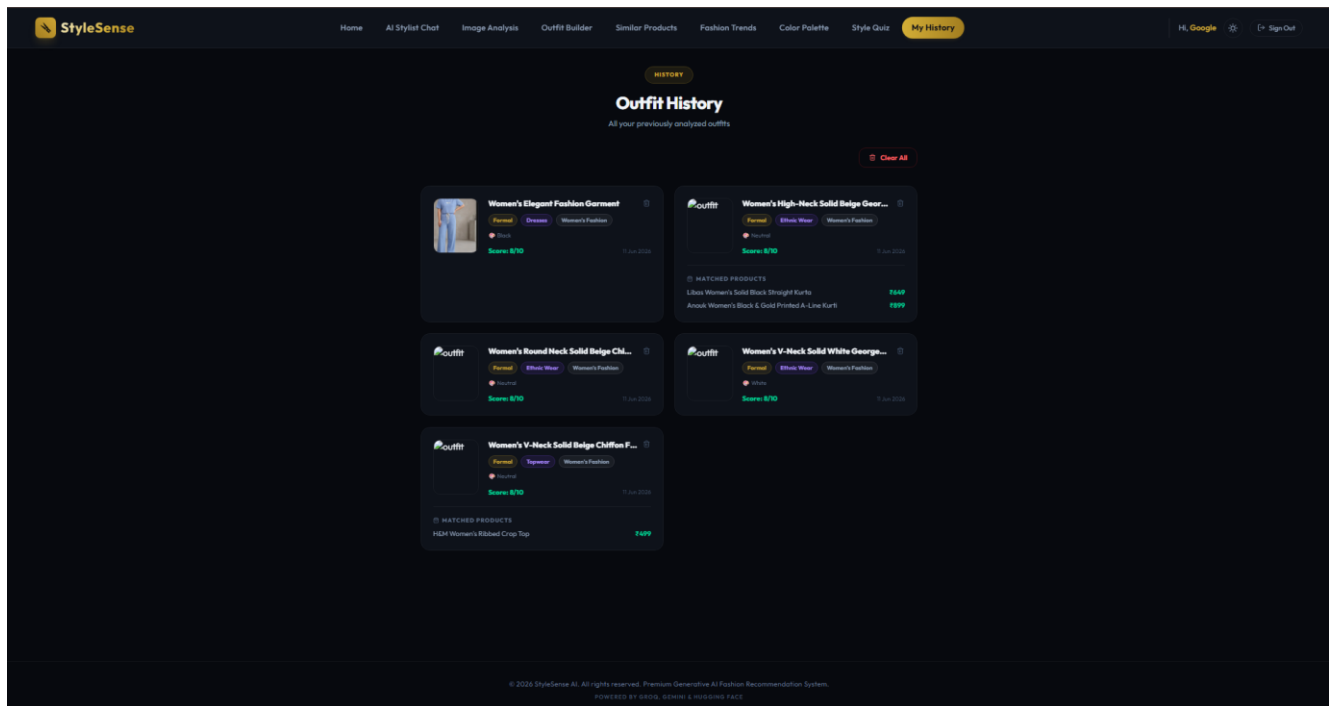
The History module contributes to a more organized and user-friendly experience by ensuring that valuable fashion insights remain accessible even after a session has ended.

Key Features:

- Storage of previous outfit recommendations
- Access to past image analysis results
- History of AI stylist conversations
- Record of style quiz responses
- Previously viewed fashion trends and insights
- Easy retrieval of past recommendations
- Personalized fashion tracking
- Organized user activity management
- Improved user experience and convenience
- Secure and efficient data management

Outcome:

The History module enables users to revisit their past fashion activities, track their style evolution, and access previously generated recommendations, creating a more personalized, organized, and seamless fashion assistance experience.



Configuration Parameters

Parameter	Value
Frontend Framework	React.js
Backend Framework	FastAPI
AI Model	Gemini 2.5 Flash
Deployment	Vercel + Render
Language	Python
Styling	Tailwind CSS

Potential Enhancements

- Virtual Try-On Feature
- User Login System
- Fashion History Tracking
- Personalized Fashion Profiles
- AR-Based Styling Experience
- Multi-Language Support
- Advanced Product Matching
- E-commerce Integration

Conclusion

StyleSense successfully demonstrates the application of Generative AI in the fashion domain. By integrating Gemini AI, image analysis, outfit recommendation, and conversational fashion assistance, the platform delivers a complete personalized styling experience. The system provides intelligent recommendations, improves user engagement, and showcases how AI can transform fashion discovery and decision-making.